

Advanced MySQL Self-Study Manual — Part 1: Concepts & Patterns

(MySQL 8+. Companion files: Part 2 — Practice DB & Exercises, Part 3 — Interview Prep)

PART 1 — FOUNDATIONS REFRESHER

You already know these; this is a fast reference, not a lesson.

Clause	Purpose
SELECT	choose columns/expressions to return
FROM	source table(s)
WHERE	row-level filter, before grouping
GROUP BY	collapse rows into groups
HAVING	filter on groups (post-aggregation)
ORDER BY	sort final result
LIMIT	cap row count
DISTINCT	remove duplicate result rows

NULL handling toolkit:

```
IS NULL / IS NOT NULL      -- the only correct way to test for
NULL
COALESCE(a, b, c)           -- first non-NULL value in the list
IFNULL(a, b)                -- MySQL shorthand for COALESCE(a, b)
-- 2 args only
NULLIF(a, b)                -- returns NULL if a = b, else
returns a
CAST(x AS SIGNED/CHAR/DATE...) -- type conversion
```

Common date/string functions you'll use throughout: DATE(), DATEDIFF(), DATE_ADD()/DATE_SUB(), DATE_FORMAT(), CONCAT(), SUBSTRING(), TRIM(), LOWER()/UPPER().

Aggregate functions: COUNT(), SUM(), AVG(), MIN(), MAX(). Remember: COUNT(*) counts rows; COUNT(column) counts non-NULL values in that column — these can differ.

PART 2 — THE SQL MENTAL MODEL: SYNTAX ORDER VS LOGICAL PROCESSING ORDER

This is the single most important mental model in SQL. Almost every “why doesn’t this work” confusion traces back to it.

The order you *type* a query (syntax order):

SELECT → FROM → JOIN → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT

The order MySQL actually *evaluates* a query (logical processing order):

FROM
↓
JOIN
↓
WHERE
↓
GROUP BY
↓
HAVING
↓
WINDOW FUNCTIONS
↓
SELECT
↓
DISTINCT
↓
ORDER BY
↓
LIMIT

Why this matters — three concrete consequences:

1. You cannot use a SELECT alias in WHERE:

```
-- ✗ fails — WHERE runs before SELECT creates the alias
SELECT salary * 1.1 AS adjusted_salary
FROM employees
WHERE adjusted_salary > 50000;

-- ✔ WHERE must repeat the expression, or you filter in
HAVING/subquery
SELECT salary * 1.1 AS adjusted_salary
FROM employees
WHERE salary * 1.1 > 50000;
```

2. You can use a SELECT alias in ORDER BY — because ORDER BY runs after SELECT:

```
SELECT salary * 1.1 AS adjusted_salary
FROM employees
ORDER BY adjusted_salary; -- ✔ works fine
```

3. Window functions see rows after WHERE/GROUP BY/HAVING but before SELECT's final projection — which is why you can't filter directly on a window function result in WHERE:

```
-- ✗ fails — window functions haven't been computed yet when WHERE
runs
SELECT *, ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC)
AS rn
FROM employees
WHERE rn = 1;

-- ✔ wrap it — filter in an outer query
SELECT * FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary
DESC) AS rn
  FROM employees
) x
WHERE rn = 1;
```

Keep this diagram in your head for the rest of the course — it explains almost every “why can't I do this” moment you'll hit.

PART 3 — ADVANCED JOIN MASTERCLASS

3.1 INNER JOIN

Returns rows where the join condition matches in **both** tables.

```
SELECT c.name, o.order_id, o.order_date
FROM customers c
INNER JOIN orders o ON o.customer_id = c.customer_id;
```

- Multiple conditions: `ON o.customer_id = c.customer_id AND o.status = 'ACTIVE'`
- Multiple joins chain left to right: `customers → orders → order_items → products`.
- **NULL behavior:** a NULL join key never matches anything, even another NULL (NULL = NULL is UNKNOWN, not TRUE) — this silently drops rows and is a common bug source.

3.2 LEFT JOIN

Keeps **all** rows from the left table, filling unmatched right-side columns with NULL.

```
SELECT c.name, o.order_id
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id;
```

Anti-join pattern (find left rows with *no* match):

```
SELECT c.*
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;  -- unmatched rows only
```

Filter placement matters enormously here: putting a right-table condition in WHERE instead of ON silently turns your LEFT JOIN into an INNER JOIN (see Part 29, Interview Traps).

3.3 RIGHT JOIN

Mirror of LEFT JOIN — keeps all right-table rows. Rarely used in practice because you can always rewrite a RIGHT JOIN as a LEFT JOIN by swapping table order; teams standardize on LEFT JOIN for readability.

3.4 FULL OUTER JOIN (simulated)

MySQL has no native FULL OUTER JOIN. Simulate with LEFT JOIN UNION RIGHT JOIN:

```
SELECT c.customer_id, c.name, o.order_id
FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id
UNION
SELECT c.customer_id, c.name, o.order_id
FROM customers c RIGHT JOIN orders o ON o.customer_id =
c.customer_id;
```

UNION (not UNION ALL) is essential here — it deduplicates the rows that matched in both directions, which would otherwise appear twice.

3.5 SELF JOIN

Join a table to itself using two aliases — classic for hierarchies and row comparisons.

```
-- Employee → manager
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.employee_id;
```

Also used for duplicate detection: join on matching business-key columns but different primary keys.

3.6 CROSS JOIN

Cartesian product — every row of A with every row of B (no ON clause).

```
SELECT p.product_name, d.date_value
FROM products p CROSS JOIN date_dimension d;
```

Useful for generating combinations (e.g., every product × every month for a report scaffold). Dangerous when accidental (forgetting a join condition) — row counts explode multiplicatively.

3.7 SEMI JOIN (via EXISTS)

A semi-join returns rows from the outer table where a match *exists*, without duplicating rows or pulling inner-table columns.

```
SELECT c.*
FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);
```

Conceptually equivalent to “customers who have at least one order,” but unlike a JOIN, a customer with 5 orders still appears **once**.

3.8 ANTI JOIN (via NOT EXISTS)

```
SELECT c.*
FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);
```

NOT EXISTS vs LEFT JOIN...IS NULL: both give the same result; NOT EXISTS is usually clearer intent and handles NULLs more safely than the NOT IN equivalent (see Part 10).

3.9 Complex multi-table joins

Build them one join at a time, verifying row counts as you go:

```
SELECT c.name, o.order_id, oi.quantity, p.product_name,
cat.category_name
FROM customers c
JOIN orders o      ON o.customer_id = c.customer_id
JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p    ON p.product_id = oi.product_id
JOIN categories cat ON cat.category_id = p.category_id;
```

Debugging tip: if row counts look wrong, comment out joins from the bottom up until the count matches your expectation, then add them back one at a time.

(Exercises for this section are in Part 2.)

PART 4 — SUBQUERY MASTERCLASS

4.1 Non-correlated subqueries

Executes independently of the outer query — one fixed value/set, computed once.

```
SELECT * FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

4.2 Correlated subqueries

References an outer-query column; conceptually re-evaluated per outer row.

```
SELECT e.* FROM employees e
WHERE salary > (
    SELECT AVG(e2.salary) FROM employees e2
    WHERE e2.department_id = e.department_id
);
```

Outer row → get department → run inner query for THAT department → compare → next row

The rule: does the comparison value change per outer row/group? Yes → correlated. Same value for everyone → non-correlated (don't add a WHERE link back to the outer table).

4.3 / 4.4 EXISTS / NOT EXISTS

Covered above as semi-/anti-join. Prefer these over IN/NOT IN when checking existence against another table, especially when NULLs may be present.

4.5 / 4.6 IN / NOT IN — and the NULL trap

```
SELECT * FROM employees
WHERE department_id IN (SELECT department_id FROM departments WHERE
active = 1);
```

The NOT IN + NULL bug:

```
-- ✗ DANGEROUS: if the subquery returns even ONE NULL, the whole NOT
IN
-- expression evaluates to UNKNOWN for every row, and the query
returns ZERO rows.
SELECT * FROM employees
WHERE department_id NOT IN (SELECT department_id FROM
employee_history);
-- ^ if this can
be NULL, bug
```

Why: $x \text{ NOT IN } (1, 2, \text{NULL})$ expands to $x \neq 1 \text{ AND } x \neq 2 \text{ AND } x \neq \text{NULL}$. The last comparison is UNKNOWN, and TRUE AND UNKNOWN is UNKNOWN — the row is excluded even when it should match. **Fix:** filter NULLs out of the subquery, or use NOT EXISTS instead (which has no such trap).

```
-- ✓ safe
WHERE department_id NOT IN (SELECT department_id FROM
employee_history WHERE department_id IS NOT NULL)
-- ✓ safer still
WHERE NOT EXISTS (SELECT 1 FROM employee_history h WHERE
```

```
h.department_id = employees.department_id)
```

4.7 / 4.8 ANY / ALL

```
salary > ANY (SELECT salary FROM employees WHERE department_id = 3)
-- greater than at least one
salary > ALL (SELECT salary FROM employees WHERE department_id = 3)
-- greater than every one (≈ > MAX)
```

4.9-4.12 Scalar / SELECT / FROM / WHERE subqueries

```
-- Scalar subquery in SELECT (must return exactly one value)
SELECT c.customer_id,
       (SELECT COUNT(*) FROM orders o WHERE o.customer_id =
c.customer_id) AS order_count
FROM customers c;

-- Derived table (subquery in FROM) – must be aliased
SELECT dept_avg.department_id, dept_avg.avg_sal
FROM (SELECT department_id, AVG(salary) AS avg_sal FROM employees
GROUP BY department_id) dept_avg
WHERE dept_avg.avg_sal > 60000;
```

4.13 Correlated vs non-correlated — comparison

	Non-correlated	Correlated
References outer query?	No	Yes
Conceptual execution	Once	Once per outer row
Typical use	Compare to one global value	Compare to a per-group value
Optimizer rewrite	N/A	Often rewritten as join internally

4.14 Subquery vs JOIN vs CTE vs window function

For “employees earning above their department average”:

```
-- Correlated subquery
SELECT e.* FROM employees e
WHERE salary > (SELECT AVG(salary) FROM employees e2 WHERE
e2.department_id = e.department_id);

-- JOIN with derived table
SELECT e.* FROM employees e
JOIN (SELECT department_id, AVG(salary) avg_sal FROM employees GROUP
BY department_id) d
ON d.department_id = e.department_id
WHERE e.salary > d.avg_sal;

-- CTE (same as JOIN, more readable for multi-step logic)
WITH dept_avg AS (SELECT department_id, AVG(salary) avg_sal FROM
employees GROUP BY department_id)
SELECT e.* FROM employees e JOIN dept_avg d ON d.department_id =
e.department_id
WHERE e.salary > d.avg_sal;

-- Window function (single pass, no self-join)
SELECT * FROM (
SELECT e.*, AVG(salary) OVER (PARTITION BY department_id) AS
dept_avg
```

```

        FROM employees e
    ) x WHERE salary > dept_avg;

```

When to use which: correlated subquery for a one-off simple check; JOIN/derived table when you'll reuse the aggregate or need best broad-compatibility; CTE when the logic has multiple readable steps; window function when you want a single efficient pass and modern MySQL 8 syntax.

PART 5 — CTE MASTERCLASS

Basic CTE

```

WITH high_value_customers AS (
    SELECT customer_id, SUM(amount) AS total_spent
    FROM orders GROUP BY customer_id HAVING SUM(amount) > 1000
)
SELECT * FROM high_value_customers ORDER BY total_spent DESC;

```

Multiple CTEs (dependency chain)

```

WITH
    orders_2025 AS (
        SELECT * FROM orders WHERE order_date >= '2025-01-01' AND
order_date < '2026-01-01'
    ),
    customer_totals AS (
        SELECT customer_id, SUM(amount) AS total FROM orders_2025 GROUP
BY customer_id
    ),
    ranked AS (
        SELECT *, RANK() OVER (ORDER BY total DESC) AS rnk FROM
customer_totals
    )
SELECT * FROM ranked WHERE rnk <= 10;

```

raw data → filtered → aggregated → ranked → final result

Each CTE can reference only CTEs defined *before* it (left to right, top to bottom).

Recursive CTE

Two parts joined by UNION ALL: an **anchor** (starting rows) and a **recursive member** (references the CTE itself, building on prior results) until it produces no new rows.

```

-- Generate numbers 1-10
WITH RECURSIVE numbers AS (
    SELECT 1 AS n -- anchor
    UNION ALL
    SELECT n + 1 FROM numbers WHERE n < 10 -- recursive step +
termination
)
SELECT * FROM numbers;

-- Employee hierarchy (org chart from the top down)
WITH RECURSIVE org_chart AS (
    SELECT employee_id, name, manager_id, 1 AS level
    FROM employees WHERE manager_id IS NULL -- anchor: the
top
    UNION ALL
    SELECT e.employee_id, e.name, e.manager_id, oc.level + 1

```

```

FROM employees e
JOIN org_chart oc ON e.manager_id = oc.employee_id -- recursive
step
)
SELECT * FROM org_chart ORDER BY level;
```

Termination happens automatically when the recursive query returns zero new rows. Always ensure a stopping condition exists (a WHERE bound, or eventually running out of matching rows) — an unbounded recursive CTE will hit MySQL's `cte_max_recursion_depth` (default 1000) and error out.

CTE vs subquery vs derived table vs temp table

Feature	CTE	Subquery	Derived table	Temp table
Readability	High (named, top-down)	Lower (nested)	Medium	High
Reusable in same query	Yes (reference multiple times)	No	No (re-declared each time)	Yes, as a stateful table
Scope	One statement	One statement	One statement	Session-wide
Materialized?	MySQL 8 may materialize or merge (optimizer decides)	Inlined	Inlined/materialized	Always materialized as a table
Indexable	No (unless MySQL materializes it — not controllable)	No	No	Yes, as a table
Best for	Multi-step readable logic	Quick one-off checks	One-time reusable in-same-query set	Large intermediate result sets across sessions

PART 6 — WINDOW FUNCTIONS

The core distinction:

GROUP BY → collapses rows (N rows in, fewer rows out)
Window Function → keeps every row (N rows in, N rows out) but adds a computed column

ROW_NUMBER() / RANK() / DENSE_RANK() — side by side

salary	ROW_NUMBER	RANK	DENSE_RANK
100	1	1	1
90	2	2	2
90	3	2	2
80	4	4	3

- ROW_NUMBER(): always unique, arbitrary tiebreak among ties.
- RANK(): ties share a rank; **leaves a gap** afterward (2, 2, 4).
- DENSE_RANK(): ties share a rank; **no gap** (2, 2, 3).

```

SELECT name, department_id, salary,
       ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary
DESC) AS rn,
       RANK() OVER (PARTITION BY department_id ORDER BY salary
DESC) AS rnk,
       DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary
DESC) AS drnk
FROM employees;

```

PARTITION BY

Splits rows into independent groups for the window calculation — like a GROUP BY that doesn't collapse rows. Omit it and the window treats the whole result set as one partition.

ORDER BY inside OVER()

ORDER BY outside OVER() sorts the *final result set*. ORDER BY **inside** OVER(...) defines the order the window function processes rows in (which matters enormously for ROW_NUMBER, running totals, LAG/LEAD) — these are two unrelated things that happen to share a keyword. There is no syntax like ORDER BY x OVER(...); OVER() is a clause attached to the function call itself: SUM(x) OVER (PARTITION BY ... ORDER BY ...).

LAG() / LEAD()

```

SELECT employee_id, effective_date, salary,
       LAG(salary) OVER (PARTITION BY employee_id ORDER BY
effective_date) AS prev_salary,
       LEAD(salary) OVER (PARTITION BY employee_id ORDER BY
effective_date) AS next_salary
FROM employee_history;

```

Month-over-month growth:

```

SELECT month, revenue,
       revenue - LAG(revenue) OVER (ORDER BY month) AS mom_change
FROM monthly_revenue;

```

FIRST_VALUE() / LAST_VALUE()

```

SELECT employee_id, effective_date, salary,
       FIRST_VALUE(salary) OVER (PARTITION BY employee_id ORDER BY
effective_date) AS starting_salary,
       LAST_VALUE(salary) OVER (PARTITION BY employee_id ORDER BY
effective_date
                                ROWS BETWEEN UNBOUNDED PRECEDING
AND UNBOUNDED FOLLOWING) AS latest_salary
FROM employee_history;

```

Trap: LAST_VALUE with the *default* frame (RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) returns the **current row**, not the true last row of the partition — you must explicitly widen the frame to UNBOUNDED FOLLOWING (see Window Frames below). This is one of the most common window-function bugs.

SUM()/AVG()/COUNT() OVER()

```

-- Running total

```

```

SELECT order_date, amount,
       SUM(amount) OVER (ORDER BY order_date ROWS BETWEEN UNBOUNDED
PRECEDING AND CURRENT ROW) AS running_total
FROM orders;

-- Percentage of total
SELECT product_id, revenue,
       revenue / SUM(revenue) OVER () * 100 AS pct_of_total
FROM product_revenue;

```

Window Frames — ROWS vs RANGE

```

ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW -- physical row
count
RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW -- logical value
range

```

The difference shows up with duplicate ORDER BY values: ROWS always looks at a fixed number of physical rows; RANGE groups *all rows with the same ORDER BY value* as a single “peer group” and includes/excludes them together. If two rows share the same order_date, a RANGE-based running total will give **both rows the same (combined) total**, while ROWS gives each its own incremental total. Default frame (if you omit the clause but have an ORDER BY) is RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW — which is why LAST_VALUE “misbehaves” unless you override it.

Advanced window patterns — quick recipes

```

-- Top N per group
SELECT * FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY category_id ORDER BY
revenue DESC) rn
  FROM product_revenue
) x WHERE rn <= 3;

-- Moving average (3-row window)
SELECT order_date, amount,
       AVG(amount) OVER (ORDER BY order_date ROWS BETWEEN 2
PRECEDING AND CURRENT ROW) AS moving_avg_3
FROM daily_sales;

-- Gaps and islands (find consecutive-date streaks)
SELECT customer_id, activity_date,
       DATE_SUB(activity_date, INTERVAL ROW_NUMBER() OVER (PARTITION
BY customer_id ORDER BY activity_date) DAY) AS island_key
FROM activity_log;
-- rows sharing the same island_key are part of one consecutive
streak

```

PART 7 — ADVANCED AGGREGATION

Conditional aggregation

```

SELECT department_id,
       SUM(CASE WHEN gender = 'F' THEN 1 ELSE 0 END) AS
female_count,
       SUM(CASE WHEN salary > 80000 THEN 1 ELSE 0 END) AS
high_earners
FROM employees GROUP BY department_id;

```

ROLLUP

```
SELECT department_id, job_title, SUM(salary)
FROM employees
GROUP BY department_id, job_title WITH ROLLUP;
```

Adds subtotal rows (department-level total, then a grand total), marked with NULL in the rolled-up columns.

GROUPING SETS / CUBE (no native MySQL support)

MySQL 8 has no GROUPING SETS or CUBE. Simulate with UNION ALL of separate GROUP BY queries, or use WITH ROLLUP for the simple hierarchical subtotal case:

```
SELECT department_id, NULL AS job_title, SUM(salary) FROM employees
GROUP BY department_id
UNION ALL
SELECT NULL, job_title, SUM(salary) FROM employees GROUP BY
job_title;
```

Pivot-style reporting

```
SELECT
    product_id,
    SUM(CASE WHEN QUARTER(order_date) = 1 THEN amount ELSE 0 END) AS
Q1,
    SUM(CASE WHEN QUARTER(order_date) = 2 THEN amount ELSE 0 END) AS
Q2,
    SUM(CASE WHEN QUARTER(order_date) = 3 THEN amount ELSE 0 END) AS
Q3,
    SUM(CASE WHEN QUARTER(order_date) = 4 THEN amount ELSE 0 END) AS
Q4
FROM order_items GROUP BY product_id;
```

PART 8 — PATTERN LIBRARY

For each: *problem* → *recognize* → *technique* → *example*.

1. Top N per group → “top 3 by X within each Y” → ROW_NUMBER() OVER (PARTITION BY group ORDER BY x DESC), filter rn ≤ N in outer query.

2. Latest/first record per group → “most recent per customer” → same ROW_NUMBER pattern, filter rn = 1.

3. Second-highest / Nth-highest salary → DENSE_RANK() OVER (ORDER BY salary DESC), filter rnk = 2 (or N). Use DENSE_RANK not ROW_NUMBER if ties should share a rank.

```
SELECT * FROM (
    SELECT *, DENSE_RANK() OVER (ORDER BY salary DESC) AS rn FROM
employees
) x WHERE rnk = 2;
```

4. Duplicate detection → “find rows with repeated business key” → GROUP BY key_columns HAVING COUNT(*) > 1.

5. Deduplication → “keep one row per key” → ROW_NUMBER() OVER (PARTITION BY key ORDER BY tiebreak), keep rn = 1, delete/ignore the rest.

6. Missing records / anti-join → “customers with no orders” → LEFT JOIN ... WHERE right.id IS NULL or NOT EXISTS.

7. Semi-join → “customers who have at least one order” (without duplicating rows) → WHERE EXISTS (...).

8. Running totals → SUM(x) OVER (ORDER BY date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW).

9. Moving average → AVG(x) OVER (ORDER BY date ROWS BETWEEN n PRECEDING AND CURRENT ROW).

10. Previous/next row comparison → LAG() / LEAD().

11. Month-over-month / year-over-year growth → LAG(value) OVER (ORDER BY period), then (current - prev) / prev.

12. Percentage of total → x / SUM(x) OVER ().

13. Gaps and islands → subtract a ROW_NUMBER() sequence from a date column; equal results = same island.

14. Consecutive dates / longest streak → island technique above, then GROUP BY island_key, take MAX(streak_length).

15. Hierarchical data / recursive hierarchy → WITH RECURSIVE anchored at root, joining child→parent.

16. Relational division (“customers who bought every product in category X”) →

```
SELECT customer_id FROM orders o JOIN products p ON p.product_id =
o.product_id
WHERE p.category_id = 5
GROUP BY customer_id
HAVING COUNT(DISTINCT p.product_id) = (SELECT COUNT(*) FROM products
WHERE category_id = 5);
```

17. Data reconciliation / comparing two datasets → FULL OUTER JOIN simulation, or two anti-joins (A-not-in-B, B-not-in-A) plus an inner join for “matched but changed” rows (compare non-key columns for differences).

18. Latest status / audit history → ROW_NUMBER() OVER (PARTITION BY entity_id ORDER BY changed_at DESC), filter rn = 1.

PART 9 — SET OPERATIONS

```
UNION          -- combines + removes duplicates
UNION ALL      -- combines, keeps duplicates (faster – no dedup pass)
```

MySQL 8.0.31+ supports INTERSECT and EXCEPT natively. On older 8.0 versions, simulate:

```
-- INTERSECT alternative
SELECT a.id FROM table_a a WHERE a.id IN (SELECT id FROM table_b);
-- EXCEPT alternative
SELECT a.id FROM table_a a WHERE a.id NOT IN (SELECT id FROM table_b
WHERE id IS NOT NULL);
```

Always prefer UNION ALL over UNION unless you specifically need deduplication — the dedup pass has a real cost.

PART 10 — NULL AND THREE-VALUED LOGIC

SQL logic isn't binary — it's TRUE / FALSE / UNKNOWN. Any comparison involving NULL evaluates to UNKNOWN, and rows where the WHERE/HAVING/ON condition is UNKNOWN are excluded (treated like FALSE for filtering, but *not* interchangeable with FALSE in boolean logic — NOT UNKNOWN is still UNKNOWN, not TRUE).

Where this bites you: - WHERE column = NULL — always UNKNOWN, never matches, even for NULL rows. Must use IS NULL. - NOT IN (subquery containing NULL) — see Part 4.6, silently returns nothing. - Aggregates **ignore** NULLs: AVG(col) averages only non-NULL values; COUNT(col) skips NULLs; COUNT(*) counts all rows regardless. - CASE WHEN column = value — if column is NULL, this branch is skipped (evaluates UNKNOWN), falls to ELSE or no match. - A JOIN ON a.key = b.key never matches two NULL keys against each other.

PART 11 — QUERY OPTIMIZATION

Indexes

An index is a separate, sorted (B-tree) structure that lets MySQL find rows without scanning the whole table — like a book's index vs. reading every page. Trade-off: indexes speed up reads but slow down writes (every INSERT/UPDATE/DELETE must also update each index) and consume disk space.

- **Primary key:** unique, non-null, and in InnoDB is the table's physical row order (clustered index).
- **Unique index:** enforces uniqueness, still speeds lookups.
- **Secondary index:** any other index; in InnoDB, a secondary index stores the primary key value and requires a second lookup ("bookmark lookup") to fetch full rows unless...
- **Covering index:** an index that contains *every column* the query needs, so MySQL never touches the underlying table row at all — the fastest possible lookup.

Composite indexes & leftmost-prefix rule

```
CREATE INDEX idx_customer_status ON orders(customer_id, status);
```

This index can serve: WHERE customer_id = ?, WHERE customer_id = ? AND status = ? — but **cannot** efficiently serve WHERE status = ? alone, because the index is sorted by customer_id first. Think of a phone book sorted by (last name, first name): you can jump straight to "Smith," and to "Smith, John" — but you can't jump straight to all "Johns" without scanning the whole book. Column order in the index must match your most selective, most-commonly-filtered-first query patterns.

Selectivity

Selectivity = (distinct values) / (total rows). A column like is_active (2 values) has terrible selectivity — an index on it barely helps, since it doesn't narrow the row set much. A column like email (near-unique) has excellent selectivity — ideal index candidate.

EXPLAIN — reading the output

Column	Meaning	Good	Concerning
type	access method	const, eq_ref, ref, range	ALL (full table scan) empty (no

possible_keys	indexes MySQL <i>could</i> use	any listed	index available)
key	index actually chosen	matches a useful index	NULL (none used)
key_len	bytes of the index used	matches expected prefix	shorter than expected (partial index use)
rows	estimated rows examined	close to rows returned	far larger than rows returned
filtered	% of examined rows that pass WHERE	high %	low % (scanning much more than needed)
Extra	extra operations	Using index (covering index — great)	Using filesort, Using temporary (expensive extra passes)

```
EXPLAIN SELECT * FROM orders WHERE customer_id = 42 AND status = 'SHIPPED';
```

EXPLAIN ANALYZE

Available MySQL 8.0.18+. Unlike plain EXPLAIN (which shows the *planned* execution and estimates), EXPLAIN ANALYZE actually **runs** the query and reports real timing and row counts per step — use it when estimates and reality seem to diverge.

Full table scans (type = ALL)

Acceptable when the table is small, or when the query genuinely needs most of the rows (no index would help). A red flag when it appears on a large table with a selective WHERE clause that *should* be using an index.

SARGability (Search ARGument ABLE)

A condition is sargable if MySQL can use an index to evaluate it directly. Wrapping the indexed column in a function breaks this:

```
-- ✗ not sargable – MySQL must compute YEAR() for every row,
ignoring any index on order_date
WHERE YEAR(order_date) = 2025

-- ✔ sargable – MySQL can seek directly within the index range
WHERE order_date >= '2025-01-01' AND order_date < '2026-01-01'
```

Same issue with WHERE UPPER(email) = 'X', WHERE salary + 100 > 5000, etc. — keep the indexed column bare on one side of the comparison.

LIKE patterns

```
LIKE 'ABC%' -- ✔ sargable – index can seek to the "ABC" range
LIKE '%ABC' -- ✗ not sargable – leading wildcard forces a full
scan
LIKE '%ABC%' -- ✗ not sargable – same problem (full-text search is
the real fix for this)
```

OR conditions

WHERE a = 1 OR b = 2 often prevents efficient single-index use (MySQL may need to scan for each branch separately or fall back to a full scan). Rewriting as UNION of two indexed queries can sometimes outperform a single OR-based scan.

IN vs EXISTS vs JOIN

- IN (small static list) — fine, often fastest.
- IN (subquery) — MySQL 8 usually optimizes this well, but EXISTS is often clearer and safer with NULLs.
- EXISTS — best when you only need existence, not any inner-table columns.
- JOIN — best when you need columns from both tables, or when reusing an aggregate across the query.

Avoid SELECT *

Pulls unnecessary columns/bytes over the wire and can prevent a covering index from applying. Name only the columns you need.

Pagination

```
-- Offset pagination – gets slower as OFFSET grows (must
scan+discard prior rows)
SELECT * FROM orders ORDER BY order_id LIMIT 20 OFFSET 10000;

-- Keyset/seek pagination – stays fast regardless of page depth
SELECT * FROM orders WHERE order_id > 10020 ORDER BY order_id LIMIT
20;
```

Keyset pagination requires a stable, indexed sort key (often the primary key or a unique timestamp) and remembering the last-seen value between pages instead of an offset count.

PART 12 — DATABASE DESIGN

Normalization

- **1NF**: atomic values, no repeating groups (no comma-separated lists in a single column).
- **2NF**: 1NF + every non-key column depends on the **whole** primary key (matters for composite keys — no partial dependency).
- **3NF**: 2NF + no non-key column depends on another non-key column (no transitive dependency — e.g., storing department_name directly on employees when it's derivable via department_id violates this).
- **BCNF**: stricter version of 3NF — every determinant must be a candidate key (handles edge cases with overlapping composite keys that 3NF misses).

Denormalization, OLTP vs OLAP

Denormalization intentionally duplicates data to avoid expensive joins — common in reporting/analytics (OLAP) tables where read speed matters more than write-time consistency. OLTP systems (order processing, banking) favor normalization for data integrity and fast, small transactional writes; OLAP systems (dashboards, data warehouses) favor denormalized, wide tables for fast aggregate reads over huge volumes.

PART 13 — TRANSACTIONS AND ACID

```
START TRANSACTION;
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;
COMMIT;    -- or ROLLBACK; on error
```

- **Atomicity:** all statements in the transaction succeed, or none do.
- **Consistency:** the database moves from one valid state to another (constraints, triggers all still hold).
- **Isolation:** concurrent transactions don't see each other's uncommitted changes (degree depends on isolation level).
- **Durability:** once committed, changes survive a crash.

SAVEPOINT lets you roll back to a mid-transaction checkpoint without discarding the whole transaction:

```
SAVEPOINT before_payment;
-- ... risky operation ...
ROLLBACK TO before_payment;
```

PART 14 — ISOLATION LEVELS

READ UNCOMMITTED → allows dirty reads
READ COMMITTED → no dirty reads, but non-repeatable reads possible
REPEATABLE READ → MySQL/InnoDB default; no dirty or non-repeatable reads; phantom reads mostly prevented (via gap locks/MVCC)
SERIALIZABLE → strictest; effectively locks reads too, transactions run as if sequential

- **Dirty read:** reading another transaction's *uncommitted* changes.
- **Non-repeatable read:** re-reading the same row twice in one transaction gives different results because another transaction committed a change in between.
- **Phantom read:** re-running the same range query twice returns a *different set of rows* (new rows appeared) due to another transaction's insert.

InnoDB specifics: InnoDB's REPEATABLE READ uses MVCC (multi-version concurrency control) — each transaction sees a consistent snapshot from its start, largely preventing phantoms for plain reads, though SELECT ... FOR UPDATE can still encounter them.

PART 15 — LOCKS AND DEADLOCKS

- **Shared (S) lock:** allows other transactions to also read, blocks writes.
- **Exclusive (X) lock:** blocks both reads-for-update and writes from others.
- **Row locks:** InnoDB locks individual rows, not whole tables — enables high concurrency.
- **Gap locks:** lock the *space between* index values to prevent phantom inserts within a range.
- **Next-key locks:** a row lock + the gap before it — InnoDB's default mechanism combining both.

Deadlock example (two sessions):

```
Session A: UPDATE accounts SET balance = balance - 10 WHERE id = 1;
-- locks row 1
```

```

Session B: UPDATE accounts SET balance = balance - 10 WHERE id = 2;
-- locks row 2
Session A: UPDATE accounts SET balance = balance + 10 WHERE id = 2;
-- waits on B's lock
Session B: UPDATE accounts SET balance = balance + 10 WHERE id = 1;
-- waits on A's lock → DEADLOCK

```

InnoDB detects the cycle and automatically kills one transaction (rolling it back) so the other can proceed. **Avoidance:** always acquire locks/update rows in a **consistent order** across all transactions (e.g., always lowest id first).

PART 16 — VIEWS

```

CREATE VIEW active_customers AS
SELECT * FROM customers WHERE status = 'ACTIVE';

```

Views provide abstraction (hide complex joins behind a simple name), security (expose only certain columns/rows to certain users), and reusability. **Misconception:** a view is not automatically faster — it's just a stored query; MySQL re-executes the underlying query each time (no native materialized views in MySQL). For a “materialized” effect, populate a real table on a schedule instead.

PART 17 — STORED PROCEDURES

```

DELIMITER //
CREATE PROCEDURE get_employees_by_dept(IN dept_id INT)
BEGIN
    SELECT * FROM employees WHERE department_id = dept_id;
END //
DELIMITER ;

CALL get_employees_by_dept(3);

```

Parameter modes: IN (input only), OUT (output only), INOUT (both). Support IF, CASE, LOOP, WHILE, cursors (row-by-row iteration), and DECLARE ... HANDLER for error handling. **Use when:** encapsulating complex multi-statement business logic close to the data, reducing round-trips. **Avoid when:** logic changes often (harder to version/test than application code), or when your team's tooling/ORM doesn't handle procedures well.

Stored Functions

Like procedures but must RETURN a single value and can be used inline in a SELECT. Must be marked DETERMINISTIC (same input → same output) or NOT DETERMINISTIC — MySQL restricts non-deterministic functions in some contexts (e.g., statement-based replication).

PART 18 — TRIGGERS

```

CREATE TRIGGER before_salary_update
BEFORE UPDATE ON employees
FOR EACH ROW
BEGIN
    INSERT INTO employee_history(employee_id, old_salary,
new_salary, changed_at)

```

```
VALUES (OLD.employee_id, OLD.salary, NEW.salary, NOW());
END;
```

OLD refers to the row before the change; NEW to the row after. Good for audit logging and validation. **Downsides:** invisible side effects (a simple UPDATE silently fires extra logic), hard to debug, can hurt write performance and hide business logic from developers reading application code. Use sparingly — audit trails and simple validation are reasonable; complex cascading business logic usually belongs in the application layer.

PART 19 — MYSQL-SPECIFIC ADVANCED TOPICS

Must know: - **InnoDB:** MySQL's default transactional storage engine; tables are clustered on the primary key (rows physically stored in PK order). - **Secondary indexes:** store PK value, requiring a lookup back to the clustered index unless covering. - **JSON columns:** products.attributes JSON; query with -> / ->> operators or JSON_EXTRACT(). - **Generated columns:** total_price DECIMAL(10,2) AS (quantity * unit_price) STORED — computed automatically, can be indexed if STORED.

Good to know: - **Full-text search:** MATCH(col) AGAINST('term') with a FULLTEXT index — solves the leading-wildcard LIKE problem for text search. - **JSON indexes:** can't index a JSON column directly, but can index a generated column extracted from it. - **Temporary tables:** CREATE TEMPORARY TABLE — session-scoped, useful for staging complex multi-step logic.

Advanced/optional: - **Partitioning:** splitting a huge table into physical sub-tables (by range/hash/list) for manageability and query pruning on very large datasets. - **Events:** MySQL's built-in scheduler for recurring stored-procedure runs. - **Cursors:** row-by-row iteration inside a procedure — a last resort; set-based SQL is almost always faster.

PART 20 — DATABASE INTERNALS (interview-level)

- **B-tree:** the structure behind InnoDB indexes — balanced tree keeping data sorted, enabling $O(\log n)$ lookups, range scans, and ordered traversal.
 - **Buffer pool:** InnoDB's in-memory cache of data/index pages — most reads hit this instead of disk if your working set fits.
 - **Query optimizer:** chooses an execution plan (which index, join order, algorithm) based on table statistics — a **cost-based** optimizer, not a rule-based one.
 - **Cardinality/statistics:** the optimizer estimates how selective each index is using stored statistics (sampled, not exact) — stale statistics after bulk loads can cause the optimizer to pick a bad plan (ANALYZE TABLE refreshes them).
 - **Why MySQL picks one index over another:** it estimates the cost (rows examined $\times$ row access cost) for each candidate plan and picks the cheapest — which is why a “logically fine” index sometimes gets ignored if statistics suggest a full scan is cheaper (common on small tables).
-

(Continue to Part 2 for the full practice database, exercises, answer key, debugging lab, and optimization lab.)